

Simple Optimizations

- Reduce hit time



Small and simple caches



Increases the miss rate

- Reduce compulsory misses



Increase the block size, prefetching



A large block size reduces the number of blocks

- Reduce capacity misses



Prefetching, better compiler algorithms

Simple Optimizations – II

- Conflict misses



- Increase the associativity



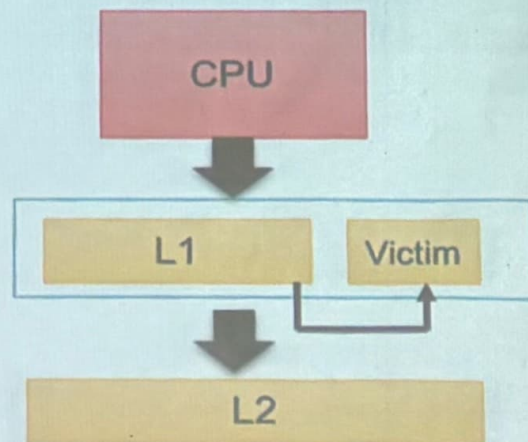
Increases the latency and power consumption



- Victim cache



The small victim cache stores frequently evicted blocks. If a block is **not found** in the L1 cache, it is **searched** in the victim cache before **searching** the L2 cache.



Static Optimizations – III

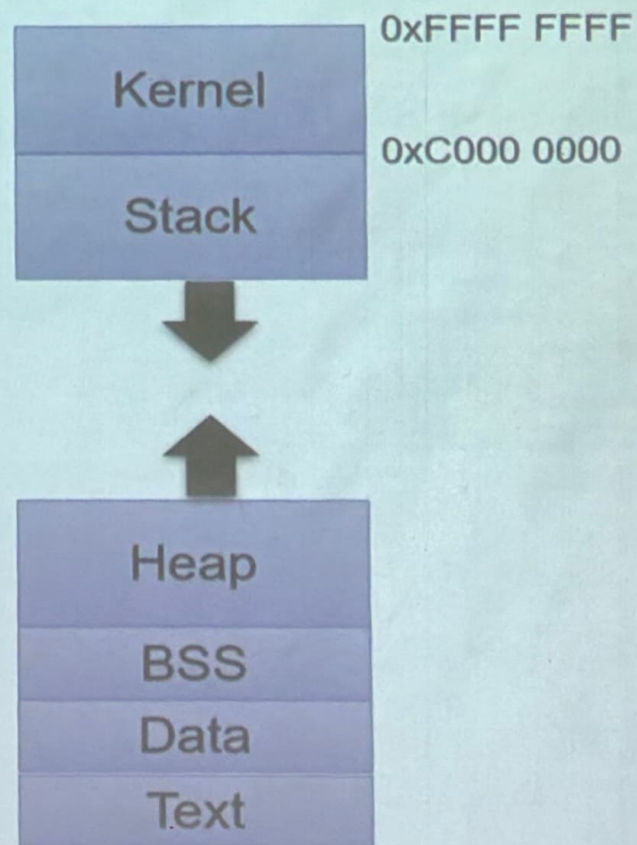
- **Reduce** the miss penalty
 - We typically **request** for 4 bytes in a 64-byte block
 - **Transfer** those 4 bytes first and then transfer the remaining 60 bytes subsequently (**critical word first**)
 - Let the processor **resume** when the 4 bytes arrive (early restart)

What does a process assume about the memory system?

Process → Running instance of a program.

It owns a large contiguous chunk of the memory space. Within this chunk it can access and modify any location at will.

Memory Map



Two Important Questions



Overlap Problem

How do we **support** multiple processes concurrently? A typical machine runs 100s of processes **concurrently**. Wouldn't they be able to **access** each other's data?



Size Problem

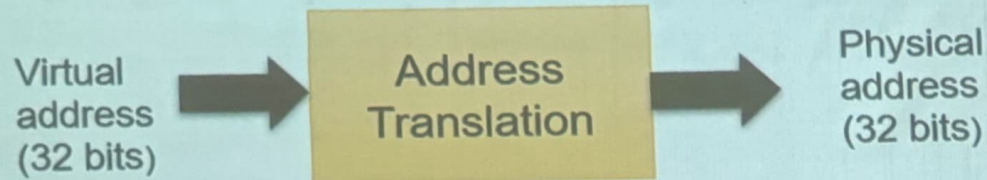
Can we **access** memory that is larger than the size of the physical memory?

Solution to the Size and Overwrite Problems

Virtual Memory

- 1 The processor generates virtual addresses
- 2 Convert virtual to actual (physical) addresses
- 3 This solves both the problems

Address Translation



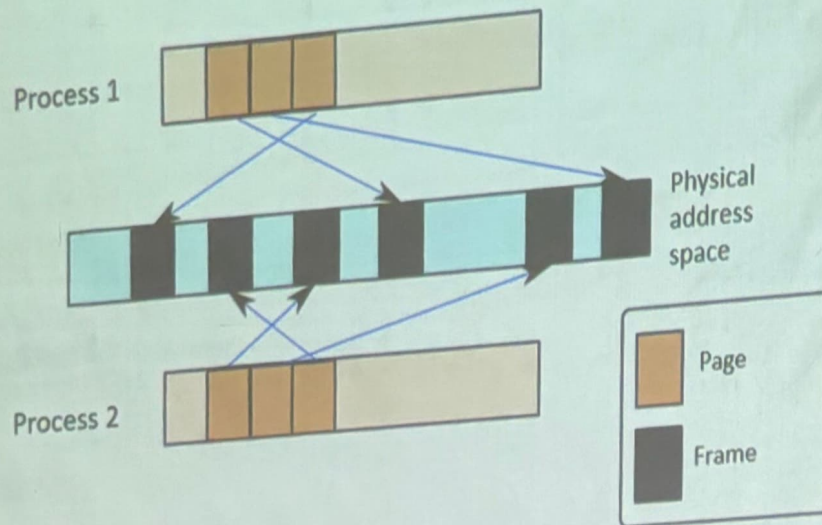
What do we **need** to ensure?

- Unless intended, two processes **never access** the same physical address
- The physical address might **point** to a location outside main memory (**solution** to the size problem)

Basic Idea

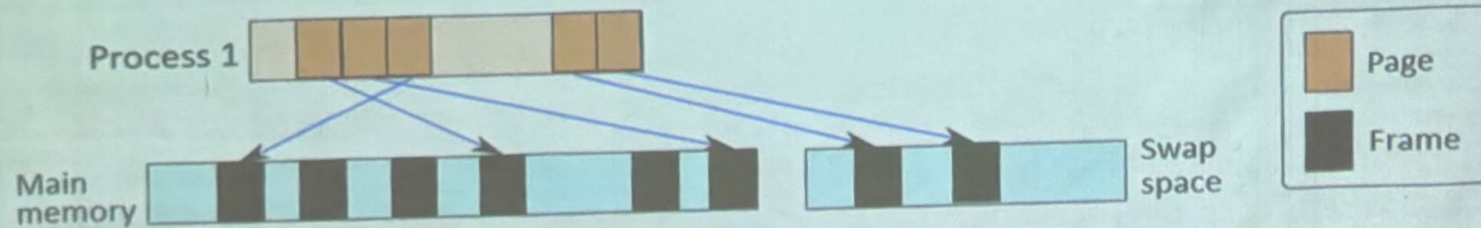
- Divide the **virtual memory** space (addresses that programmers and compilers see)
 - Into 4 KB pages
- Divide the physical memory into 4 KB frames
- Map pages to frames
- Ensure that the **mapping** solves the size and overwrite problems

Overlap Problem: Map in such a way that there are no overlaps



Map pages to frames such that there are no **overlaps**.

Size Problem: Create an array of frames in the hard disk called swap space

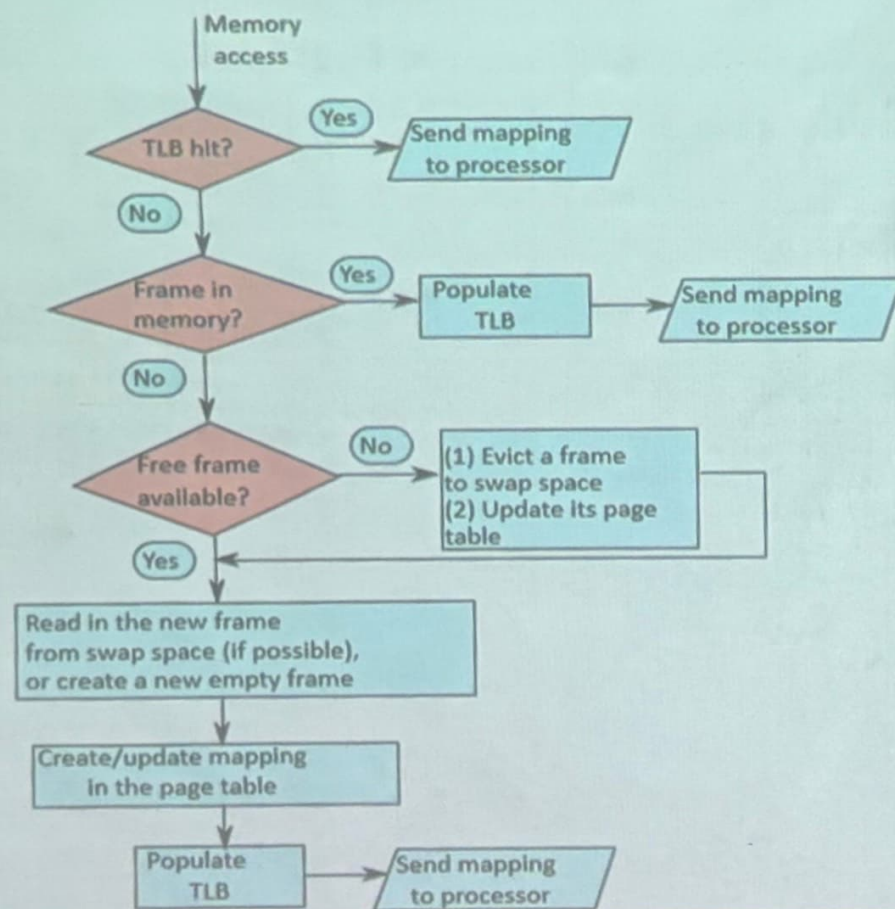


- **Create** an array in the hard disk or other form of **stable storage**. This is the **swap space**.
- If a page **points** to a frame in the swap space, we need to **bring** the frame to main memory.

Page Tables and TLBs

- The mappings are stored in a dedicated data structure called the **page table**.
- The page table takes several cycles to access. This is a **slow process**.
- Have a fast hardware structure called the TLB to **cache** the most frequent mappings
- The processor **accesses** the TLB first, **uses** the mapping if it is there. Otherwise, it **accesses** the page table.

Flowchart for a memory access



Contents



1. Overview of Memory Systems

2. Modelling Caches

3. Advanced Cache Design

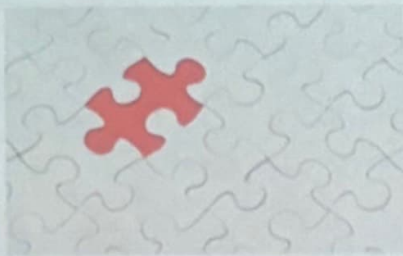
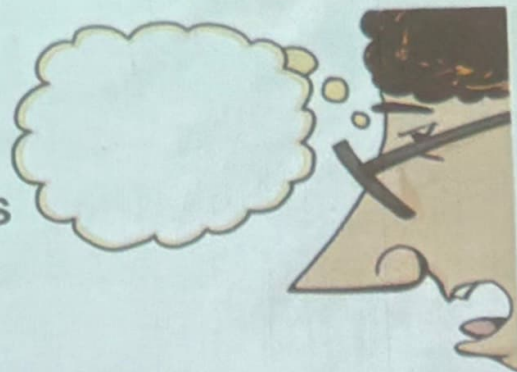
4. Trace Caches

5. Instruction Prefetching

6. Data Prefetching

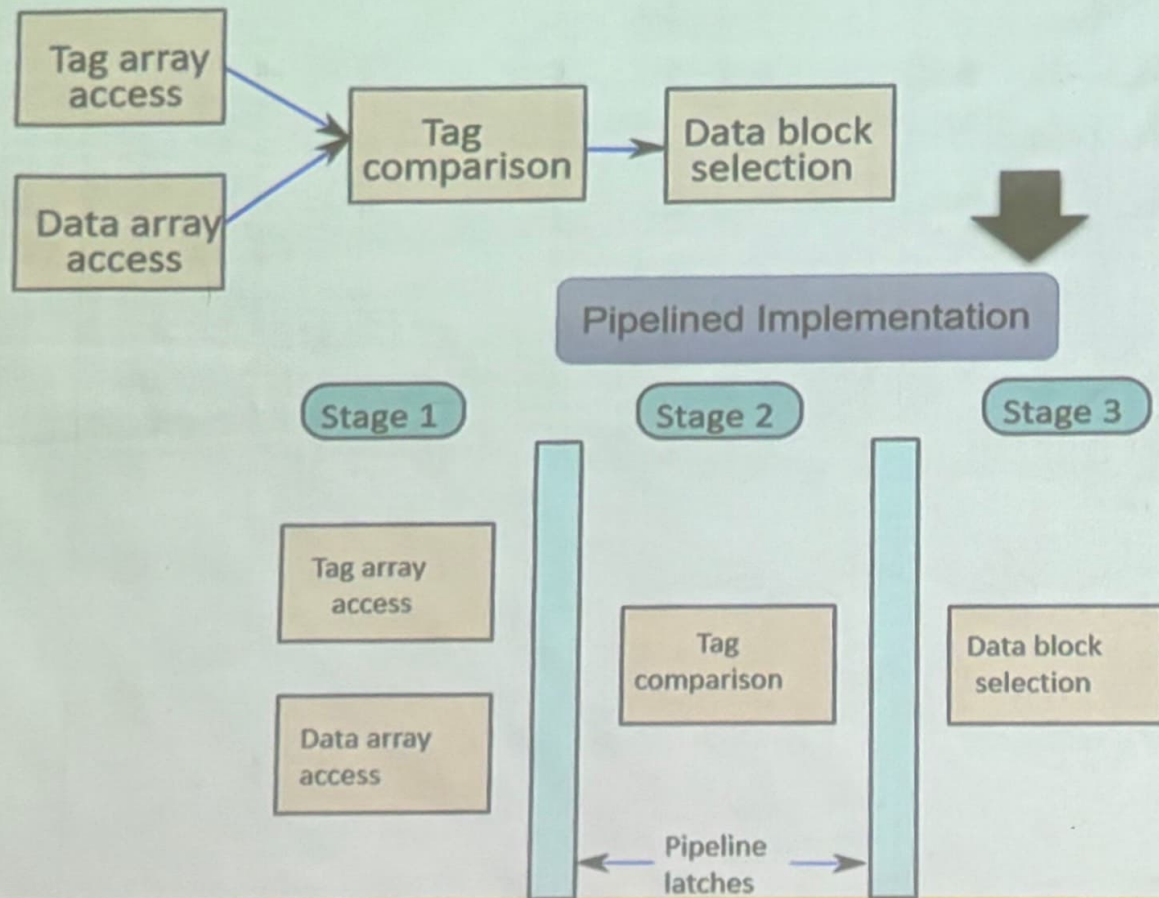
Pipelined Caches

- Let us say that a cache takes 5 cycles to **access**.
- Do we wait for 5 cycles for the **next access**?



- **Pipeline** the cache.
- Even if the **latency** is **high**, we can achieve a **high** throughput.

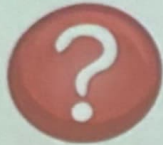
Tasks Involved in a Read Operation



More on Pipelining

- We simply **add** new pipeline buffers
- The SRAM **access** can be pipelined by adding latches after the decoder
 - This is very **expensive** in terms of area and power.
- The **write access** can be **pipelined**
 - Access tag array → Compare the tags → Effect the write in the data array

Non-Blocking Caches



- Let us say that there is a **cache miss**.
- Do later cache **access** stall?
- Cache **accesses** need not be **blocked**. We simply need to record all accesses to the **missed** block.



MSHR

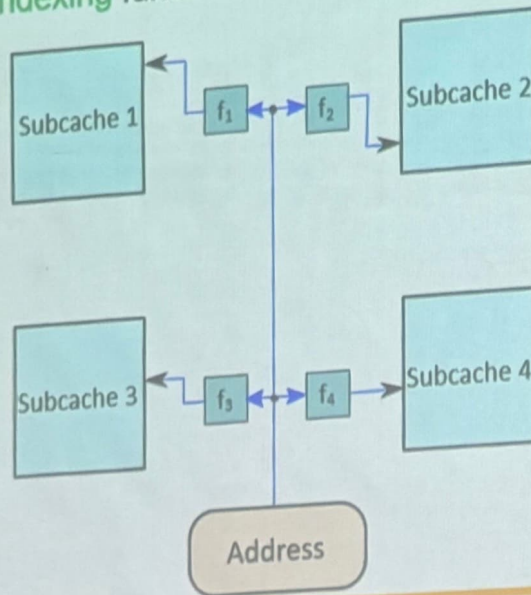
Miss Status Holding Register

Operation of an MSHR

- When there is a **miss** in a cache, an entry is **created** in its MSHR. If there are no **free** entries, the request blocks.
- We also **create** an entry in the miss queue.
- This is a **primary miss**.
- If at the same time, another miss arrives, it will be called a **secondary miss**.
- If it is a **write**, we just append it to the tail of the miss queue.
- If it is a read, we **search** for earlier writes in the miss queue to the same set of bytes. If we **find** an entry, the value is **forwarded**. Else, we enqueue the entry.

Skewed Associative Caches

- **Conflict misses** are the main problem
- **Solution:**
 - Have **multiple** subcaches
 - Use **different indexing** functions for each subcache



Basic Insight

Consider two addresses

- If $f_1(A_1) = f_1(A_2)$ **most likely** $f_2(A_1) \neq f_2(A_2)$
- If they **conflict** in one subcache most likely they will not conflict in another subcache
- Reduce conflict misses

Overheads

- These functions need to be computed
- Replacements are tricky



Assume that we want to **add** the block B at address A

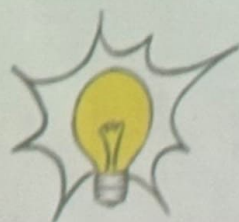
- We will first **check** if $f_1(A) \dots f_4(A)$ are free
- If not, we **evict** the block B' with address A' stored at $f_1(A)$. We insert block B in the first subcache at $f_1(A)$.
- We try to **accommodate** B' at $f_2(A')$ or $f_3(A')$ or $f_4(A')$
- If none of these locations are **free**, we **choose** one of the locations **randomly**. Let's say, it is $f_2(A')$. We insert B' there by **evicting** block B'' . Again we try to **accommodate** B'' .
- We **repeat** this process a few times before finally giving up and evicting a block.

Way Prediction

- **Default** mechanism in set-associative caches
- **Read** all the tags and blocks in a set (**simultaneously**)
- **Compare** the tag part of the address with each of the tags



- We do **more work** than what is required.



- **Predict** the way in which the tag will be found
- **Access** that way first
- If the tags don't **match**, access the rest of the ways
- **Fast and power-efficient**

Way Prediction Techniques

If the address is **available** use it, else use some other piece of **information**.

Approach 1

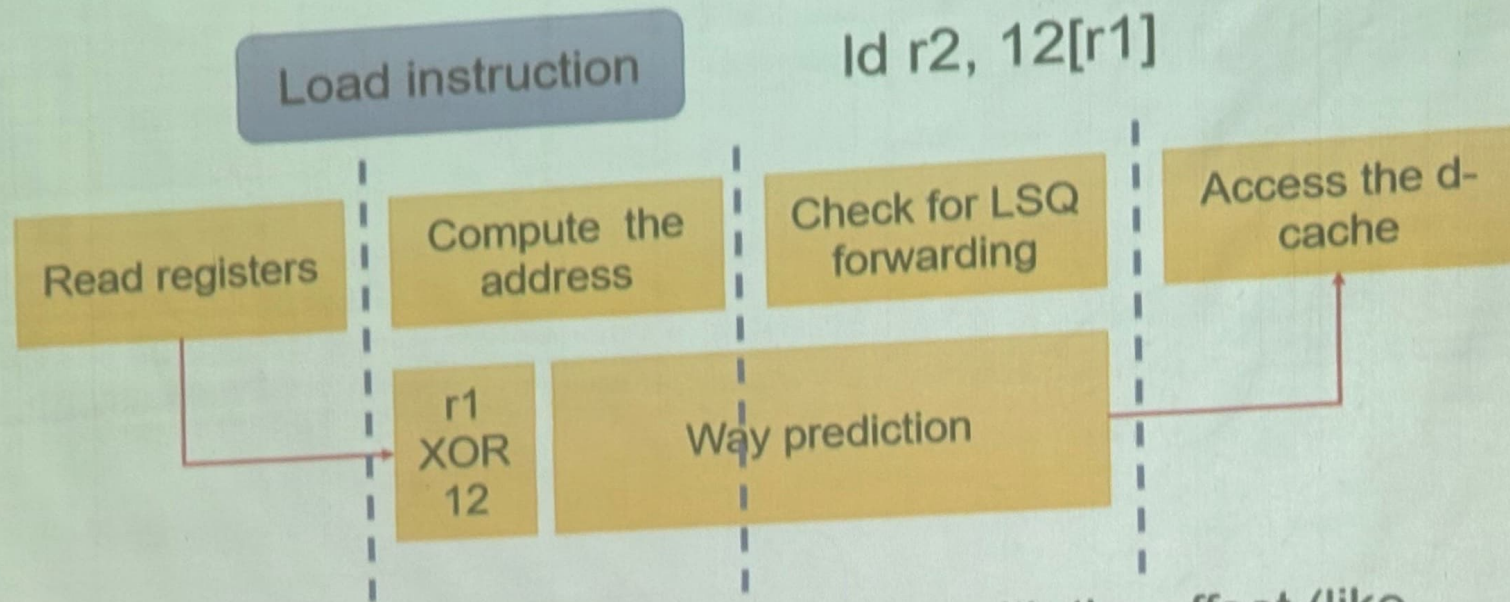
- **Predict** based on the program counter
- **Use** a branch predictor-like mechanism
- **Problems:** Memory addresses for an instruction might keep changing

Approach 2



Use an estimate of the memory address to predict the way

Way Prediction Techniques – II



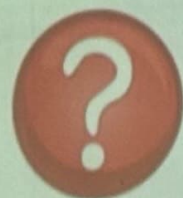
- **Combine** the value of the base address with the offset (like GShare). We may **use** additional PC bits
- Then **use** a branch predictor-like mechanism to **predict** the **way**

Loop Tiling



How do we **run large loops** that access large arrays on a system with **caches**?

How is a 2D array **stored**?



- **Row-major** order: Store the first row, then the second, then the third and so on.
- **Column-major** order: first column, second column, and so on

Basic Matrix Multiplication

Matrix Multiplication

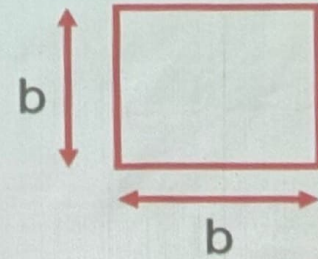
```
for (i=0; i<N; i++) {  
    for (j=0; j<N; j++) {  
        sum = 0;  
        for (k=0; k<N; k++)  
            sum += A[i][k] * B[k][j];  
        C[i][j] = sum;  
    }  
}
```



Not suitable when arrays are large, and caches are comparatively much smaller.

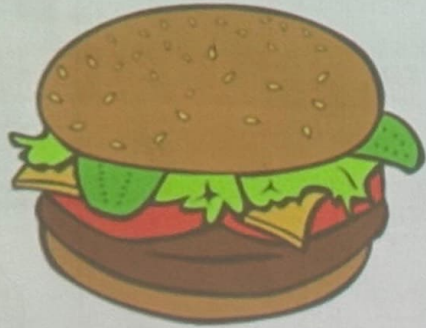
Tiled loops

```
/* Iterate through the tiles */  
for (ii = 0; ii < N; ii += b) {  
    for (jj = 0; jj < N; jj += b) {  
        for (kk = 0; kk < N; kk += b) {  
  
            /* iterate within a tile */  
            for (i = ii; i < (ii + b); i++) {  
                for (j = jj; j < (jj + b); j++) {  
                    for (k = kk; k < (kk + b); k++) {  
                        C[i][j] += A[i][k] * B[k][j];  
                    }  
                }  
            }  
        }  
    }  
}
```



Fits within the
cache

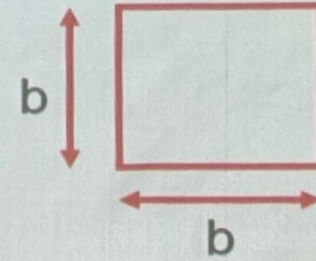
...



Prove that the algorithm is correct.

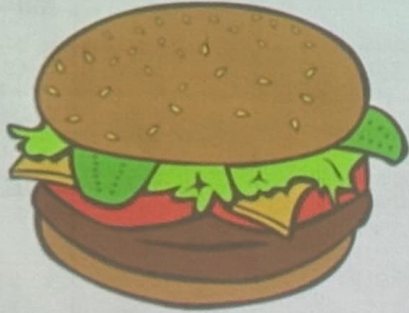
Tiled loops

```
/* Iterate through the tiles */  
for (ii = 0; ii < N; ii += b) {  
    for (jj = 0; jj < N; jj += b) {  
        for (kk = 0; kk < N; kk += b) {  
  
            /* iterate within a tile */  
            for (i = ii; i < ( ii + b ); i++) {  
                for (j = jj; j < ( jj + b ); j++) {  
                    for (k = kk; k < ( kk + b ); k++) {  
                        C[i][j] += A[i][k] * B[k][j];  
                    }  
                }  
            }  
        }  
    }  
}
```



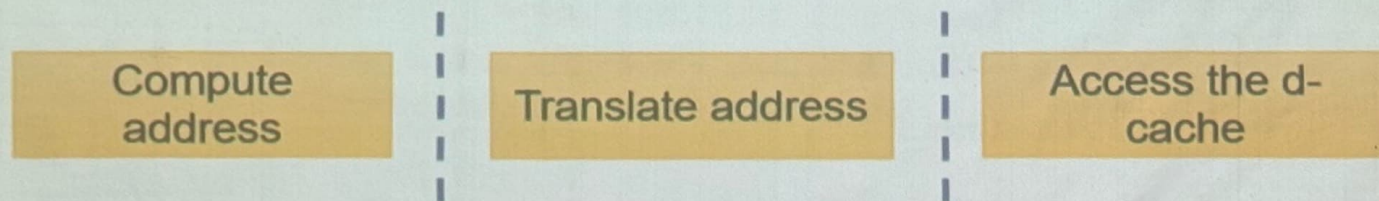
Fits within the
cache

...



Prove that the algorithm is correct.

VIPT (Virtually Indexed Physically Tagged) Caches



- We need to **spend** an additional cycle translating the address after we have **computed** it.



What if we don't need any **translation** to access the correct set in the L1 cache?

Possible

VIPT Caches

Virtual address

Page id

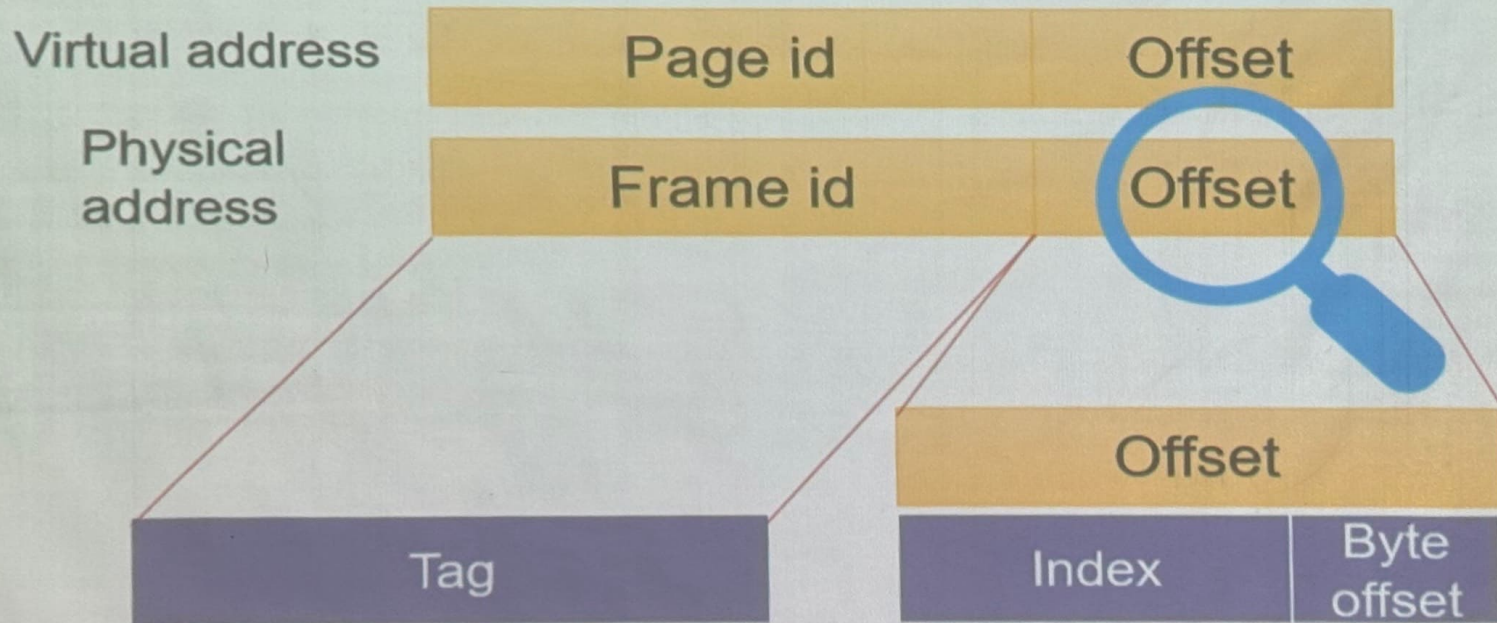
Offset

Physical
address

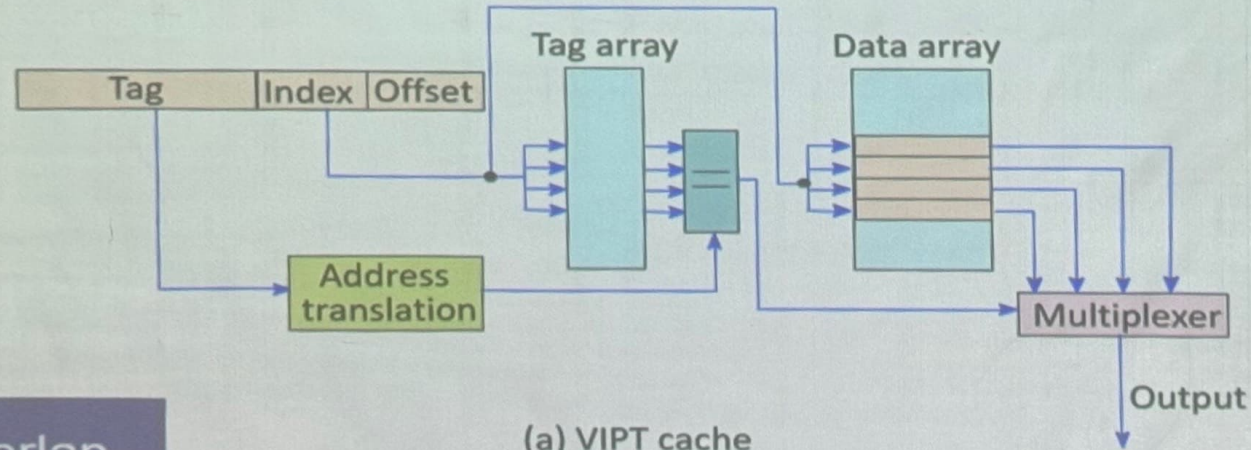
Frame id

Offset

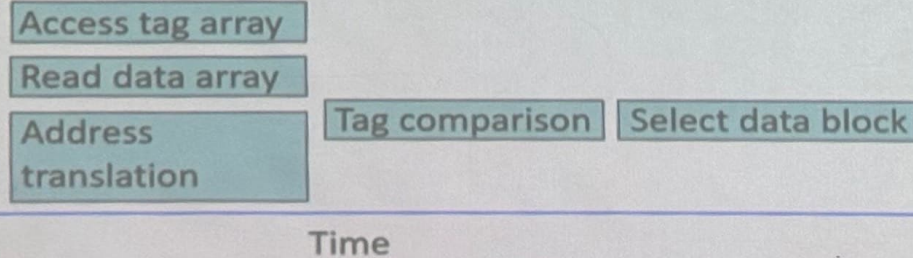
VIPT Caches



The Design of the VIPT Cache



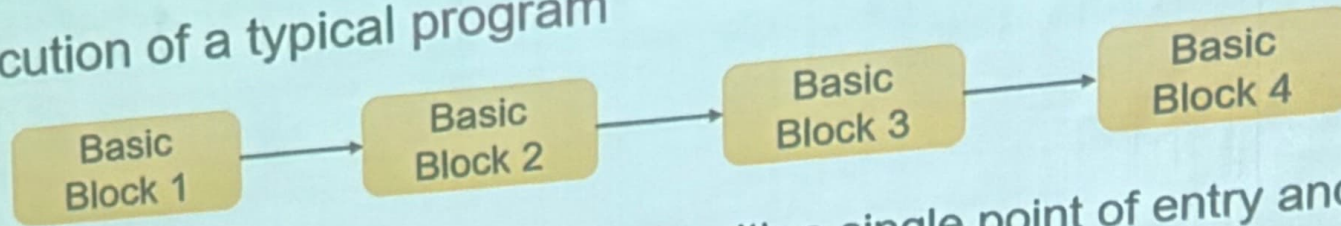
Overlap



(b) Timeline

Trace Cache

Execution of a typical program



- Basic block → A set of instructions with a single point of entry and a single point of exit
- We fetch a **sequence** of basic blocks from the i-cache
- Such a sequence is called a trace
- Let us have a cache that stores such **traces**. We have the option of sequentially reading out a full **trace**
- This is called a trace **cache** (first major **use**: Pentium 4, circa 2000)
- If the trace is **accurate**, we need not **predict** branches or decode instructions

Early Approaches

Traditional approach

- A cache line contains **contiguous** cache blocks

1

Peleg and Weiser

- Store **successive** basic blocks per cache line
- Trace segments cannot span multiple cache lines

2

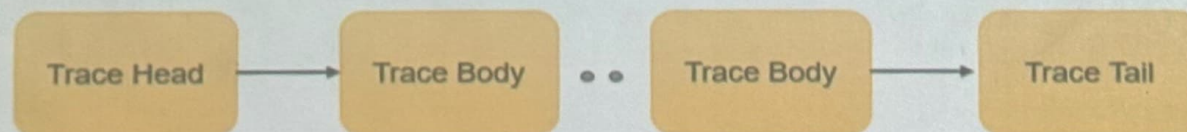
Melvin et al.

- CISC instruction sets **decode** instructions into micro-ops
- The idea is to store **decoded** micro-ops for each instruction in a cache line
- Saves some decoding effort

Can we **combine and augment** these solutions?

Structure of a Solution

- A trace consists of **multiple** cache lines
- It is a linked list of **cache lines** (each line is a trace segment)
- We need a method to place a marker to **terminate** a trace and **start** a new trace.



Basic Rules

1

Rules for storing **trace segments** in a data line

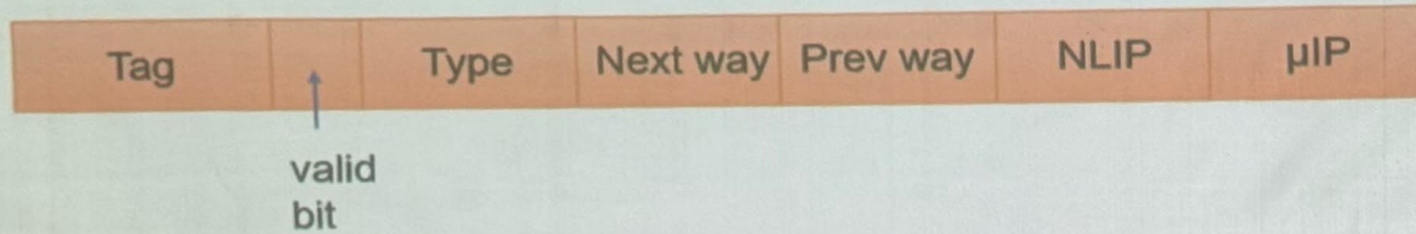
- Never **distribute** micro-ops of a macro instruction across cache lines
- **Terminate** a data line if you encounter more branch micro-ops than a threshold

2

Termination of the trace creation process

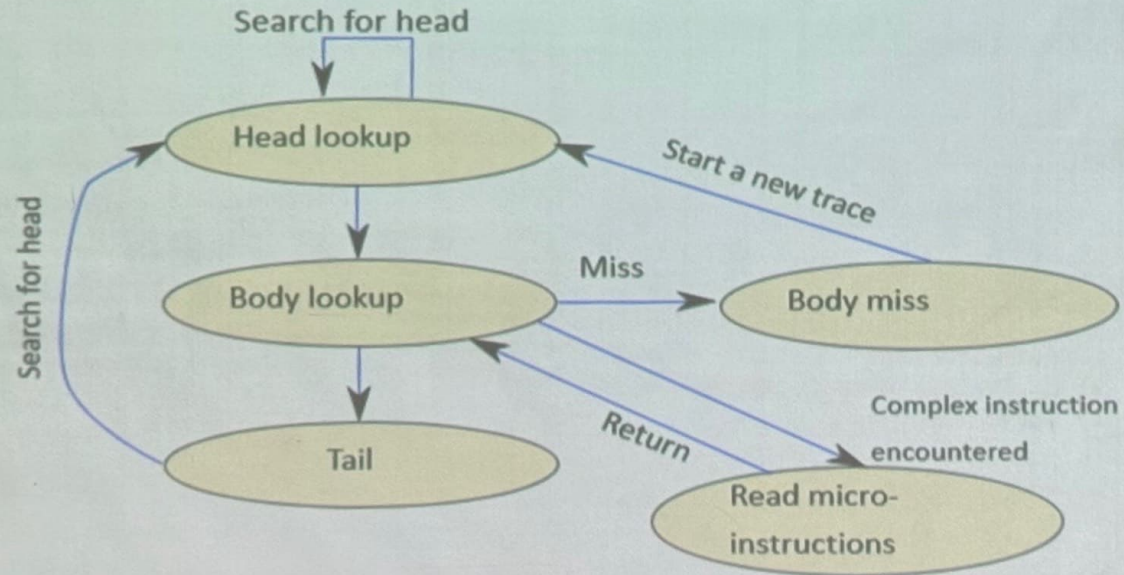
- Encounter an **indirect branch**
- **Interrupt** or branch **misprediction** notification
- **Maximum** length of trace (64 sets) reached

The Tag Array



Field	Meaning
Type	Trace head, body, or tail
Next way	Index of the way for the next trace segment (next set)
Prev way	Index of the way of the previous trace segment (prev. set)
NLIP	Address of the next CISC instruction
μIP	Index into the table of micro-instructions

State Diagram (reading a trace)



Contents

1. Overview of Memory Systems

2. Modelling Caches

3. Advanced Cache Design

4. Trace Caches



5. Instruction Prefetching

6. Data Prefetching

Misses in the Caches

We can incur a **large** performance penalty if there is a **miss** in the i-cache

- For the next 10-50 cycles, there will be no **instructions** to fetch, if there is an **L2 hit**
- If there is a **miss** in the L2, we have nothing to do for **hundreds** of cycles
- **IPC** will **suffer**

What is the **solution**?

- **Prefetch** memory addresses
- **Meaning:** **Predict** memory addresses that will be accessed in the **future**. Fetch them from the lower levels of the **memory** hierarchy before they are actually required.

Instruction Prefetching



- Find patterns in the i-cache access sequence
- Leverage this pattern for prefetching



Next Line Prefetching

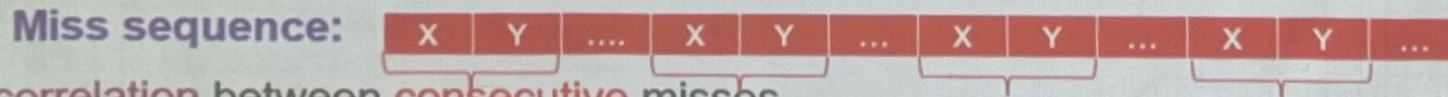
- **Pattern: Spatial locality**
- If cache line X is accessed, there is a high probability of accessing lines: $X+1$ and $X+2$
- **Reason:** The code of most functions spans **multiple** i-cache lines and we have spatial locality
- **Leverage** this pattern: If a cache line X incurs an i-cache miss, read X and the next k lines from the L2 cache
- If k is too **high**, we might fill the cache with useless data.



Almost all **prefetching** algorithms operate on the **miss** sequence, not on the **access** sequence.

Markov prefetching

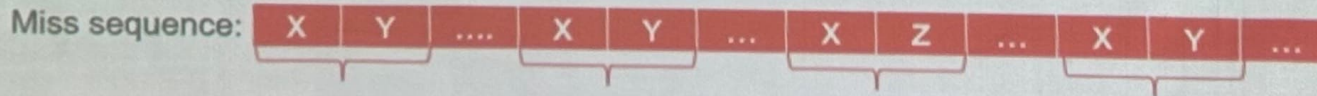
- **Pattern:** i-cache miss sequences have high repeatability
- The **miss sequence** of a core typically looks like this:



- High **correlation** between **consecutive** misses
- **Leverage** this pattern for prefetching

Markov prefetching

- Record the i-cache miss **sequences** in a table
- Given that cache line *X* incurs a **miss**, **predict** the line which will incur a miss next

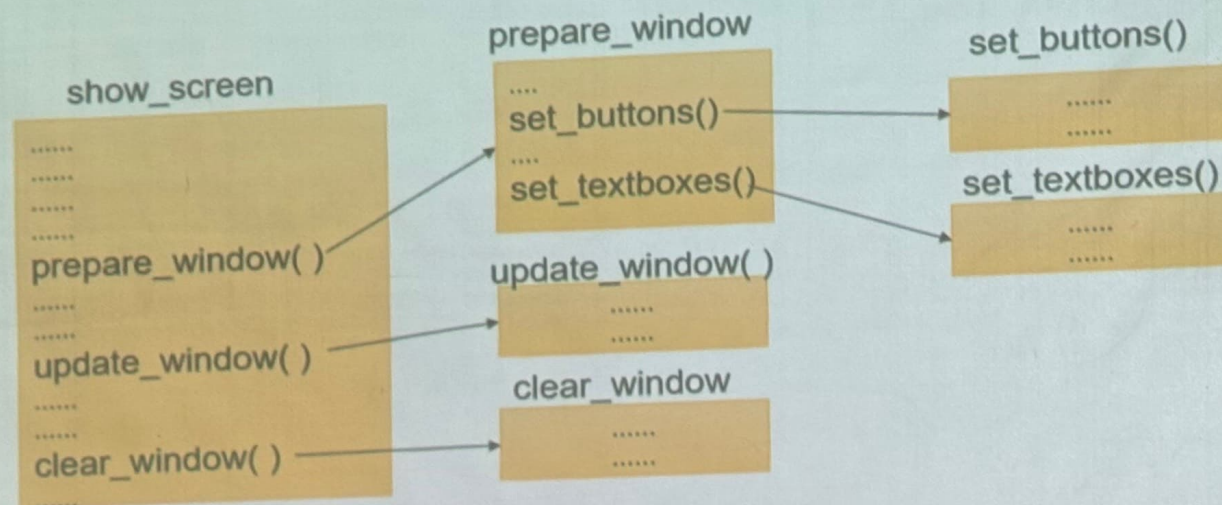


Markov table:

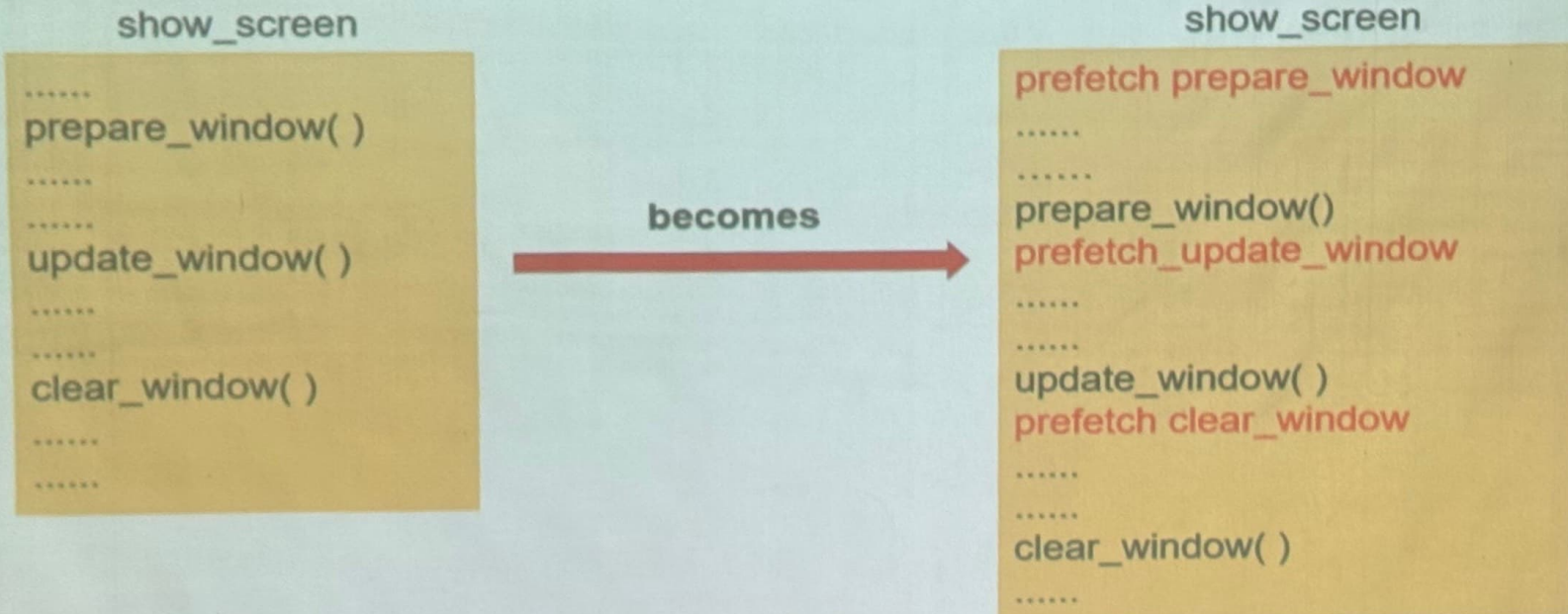
Call Graph Prefetching

- **Pattern:** The function call sequence is predictable
- **Leverage** this pattern: predict and prefetch the function that may be called next

Call Graphs

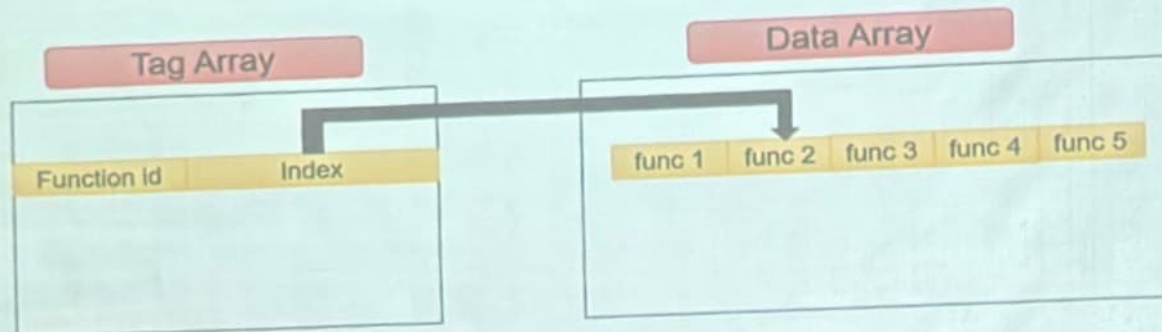


Call Graph based Prefetching (in software)



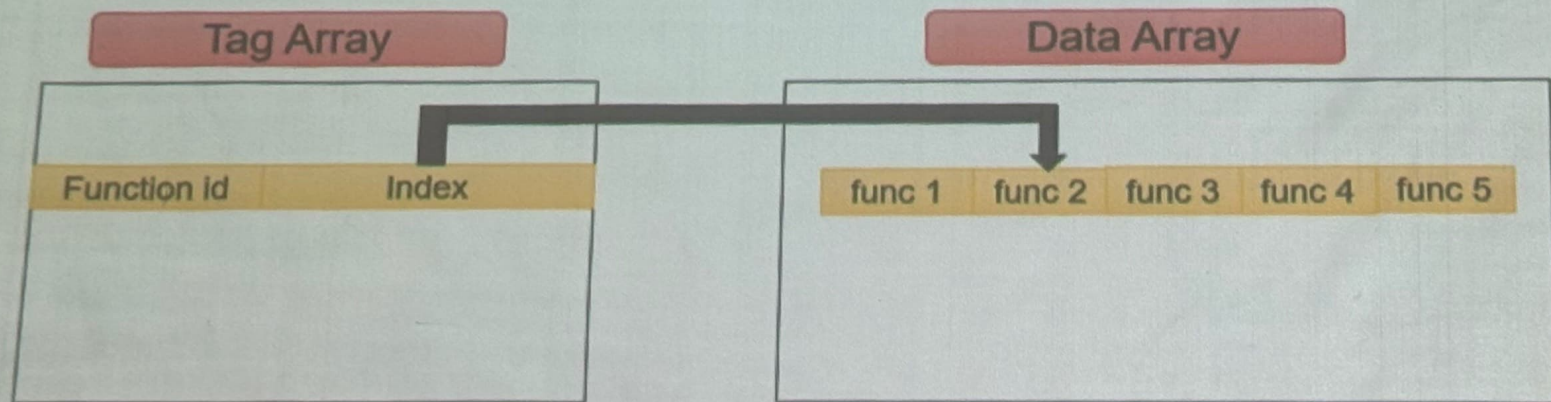
The compiler analyzes the code and inserts instructions to prefetch the code of functions.

Hardware Approach Call Graph History Cache (CGHC)



- Each **entry** in the data array contains a list of **functions** to be subsequently executed.
- The **index** maintains a **pointer** to a function in the **data** array entry.
- The index is initially 1. We **prefetch** the code of *func1*. After **returning** from *func1*, we set the index to 2, and **prefetch** the code for *func2*.

Hardware Approach Call Graph History Cache (CGHC)



- Each **entry** in the data array contains a list of **functions** to be subsequently executed.
- The **index** maintains a **pointer** to a function in the **data** array entry.
- The index is initially 1. We **prefetch** the code of *func1*. After **returning** from *func1*, we set the index to 2, and **prefetch** the code for *func2*.

Data Prefetching

Instead of **instructions**, let us now **prefetch** data



Important distinction

- **Instructions** are fetched into the in-order part of the OOO pipeline
- **Data** is fetched into the OOO pipeline
 - As a result, the final IPC is slightly more resilient to data cache misses
 - Nevertheless, prefetching is **very useful**
 - i-cache hit rates are **typically** very high, whereas d-cache hit rates are much lower
 - Hence, the margin for improvement is **significant**.

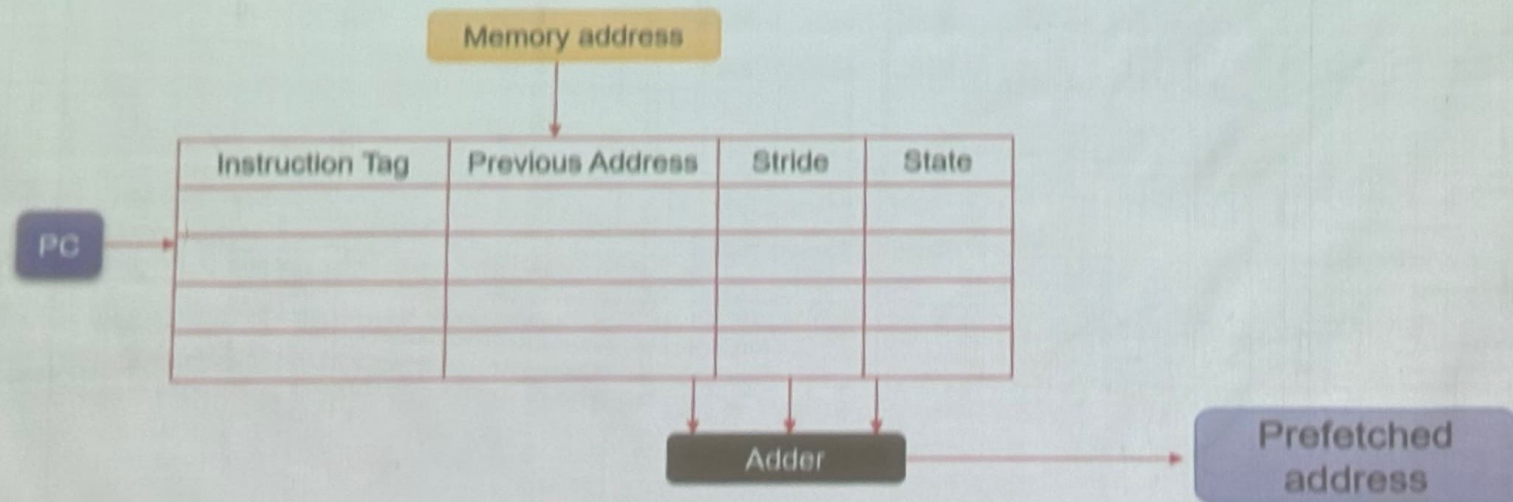
Stride based Prefetching

```
for (i = 0; i < N; i++ ) {  
    ...  
    A [i] = B[i] + C[2*i];  
}
```

Consider the following piece of code

- For the arrays *A* and *B*, the **addresses** in each **iteration** differ by 4 bytes (assumed to be the **size** of an integer)
- For the array, *C*, consecutive accesses differ by 8 bytes
- This instruction will **translate** into multiple load/store RISC instructions
 - There will be only one **memory** access per instruction
- The hardware will observe that for the same **PC**
 - The memory **address** keeps getting incremented by a certain value (4 or 8 bytes in this case)
 - This fixed increment is called a **stride**

Reference Prediction Table (RPT)



For each **instruction**

- Keep track of the **address** and the **stride**
- We can decide to **prefetch** **N iterations** in advance
- **State**: initial $\rightarrow$ transient (not sure about the **stride**) $\rightarrow$ steady

Extensions



How many iterations do we prefetch in advance?

- This depends upon the rest of the instructions in the *for* loop
- This ideally should be *dynamically* adjusted
- For a subset of *prefetches*
 - Monitor the *number* of cycles between when a line is *prefetched* and it is actually *used*
 - Should not be negative
 - It is being prefetched too late. We will not get the desired benefits.
 - Should be a *small* positive number
 - The data arrives just *before* it is needed.

Pointer Chasing

```
...  
/* traverse the linked list */  
node * temp = start_node ;  
while ( temp != NULL ) {  
    prefetch (temp -> next ); /* prefetch the next node */  
    process ( temp );  
    temp = temp -> next ;  
}
```

- Linked list **accesses** cannot be characterized by strides.
- We can insert code to **prefetch** subsequent linked list nodes.

Runahead Mode

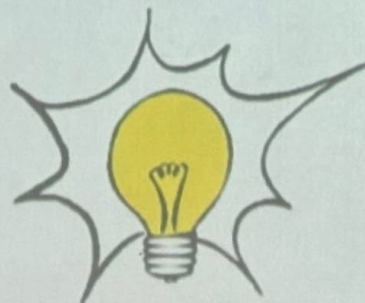
What happens when we do incorrect **prefetching**?

- We have **misses** because we do not prefetch the correct data and we **displace** useful data. L1 misses can more or less be taken care of by an OOO pipeline
- L2 **misses** are bigger problems

What happens on an L2 miss

- We go to **main memory** (200-400) cycles
- What does the OOO pipeline do?
 - The IW and ROB fill up
 - It pretty much stalls

Idea: Do some work during the stalled period



What can we do?

Enter **runahead** mode

- Return a *junk* value for the request that misses in the L2 cache
- **Restart** execution with the *junk value*
- We will produce junk **values** (in the forward slice)
- That is still okay. Why?
 - We will have a lot of instructions in the **pipeline** that can be executed correctly because they are not in the **forward slice** of the mispredicted load.
 - We will **prefetch** data into the caches, and train the predictors

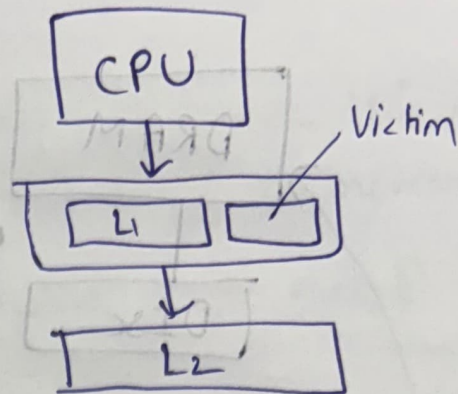
Once when the L2 miss **returns**

- Flush the **instructions** in the forward slice of the L2 miss
- **Re-execute** them
- Very **effective** prefetching technique

22 sept

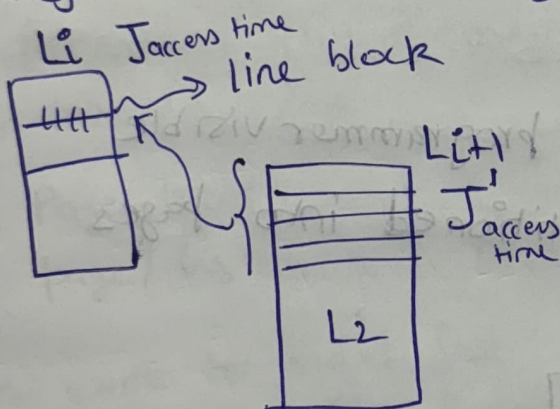
Simple optimisations

- Reduce hit time
- Reduce Capacity Miss
- Reduce Conflict misses
 - increase associativity
 - Victim cache



Ex 0 2 0 2 ... 19
Victim

- Reduces miss penalty
 - we typically request for 4 bytes in a 64 byte block transfer 4 then remaining 60 bytes



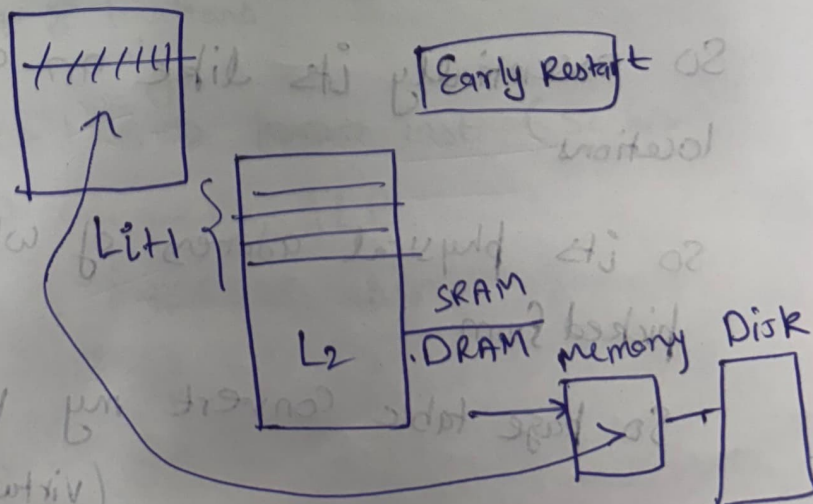
Do you wait for All 64 bytes or is it better to transfer first 4 bytes so CPU can start working and then Simultaneously Next lines keep coming

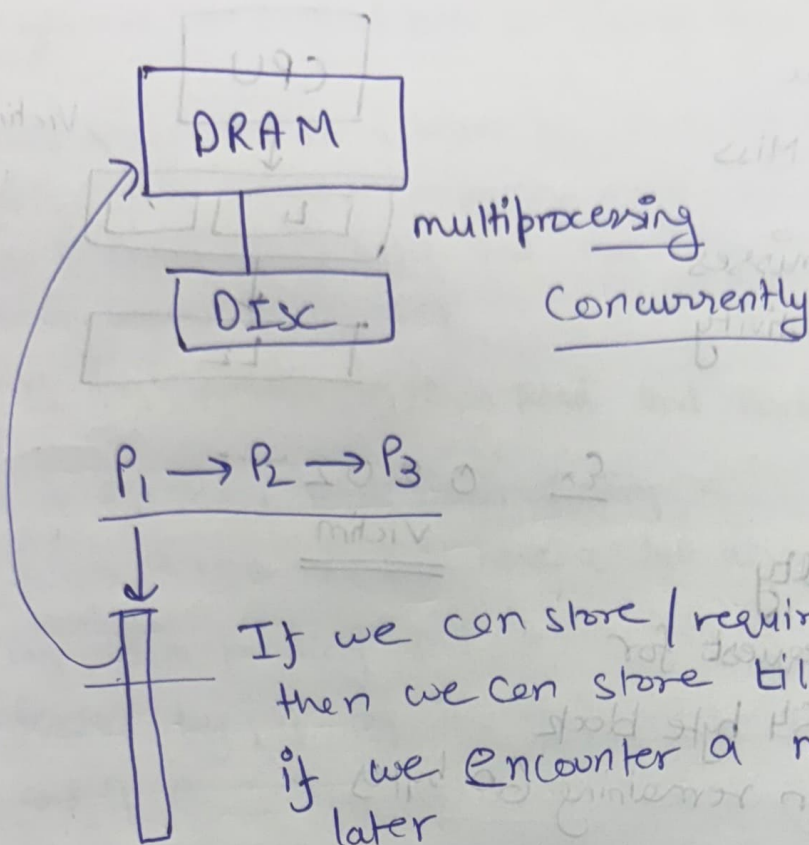
- Let processor resume when 4 bytes come (Early Restart)

$$T_i < T_{i+1}$$

$$\text{Size}_i < \text{Size}_{i+1}$$

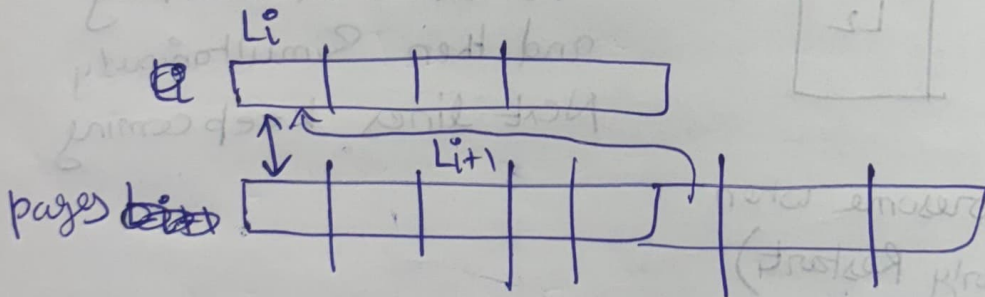
$$\text{Cost}_i > \text{Cost}_{i+1}$$





If we can store / require at a time then we can store till here / only. if we encounter a miss we can fetch later

Now I want to make it programmer visible
we say disc is partitioned into pages



this now becomes under programmers control

So essentially its like an array with $2^{32} / 2^{64}$ locations

so its physical address of where it has to be picked from

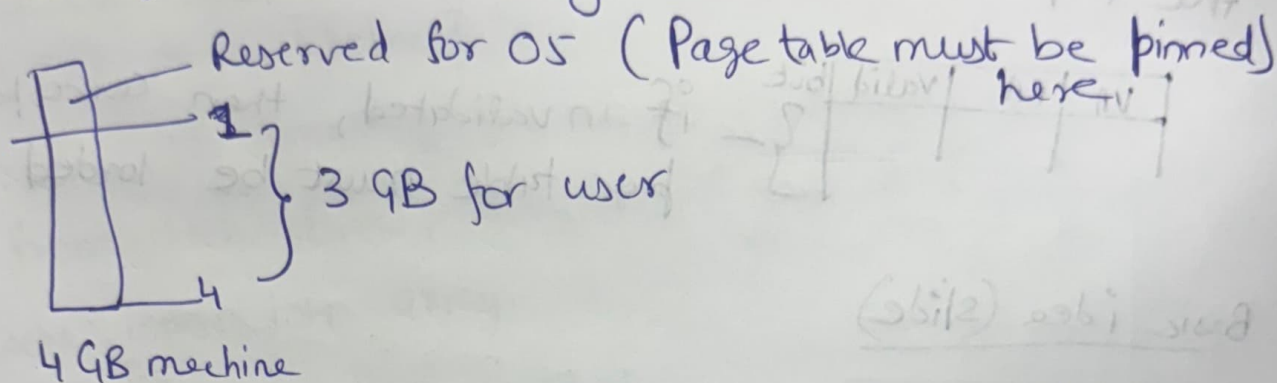
So page table convert my V.A to PA mapping

(Virtual Add) (Physical Add)

So with 32 bit address space we can have 2^{32} bits to places.

So this P.T. must be in memory (because this is what we need to look before searching in memory)

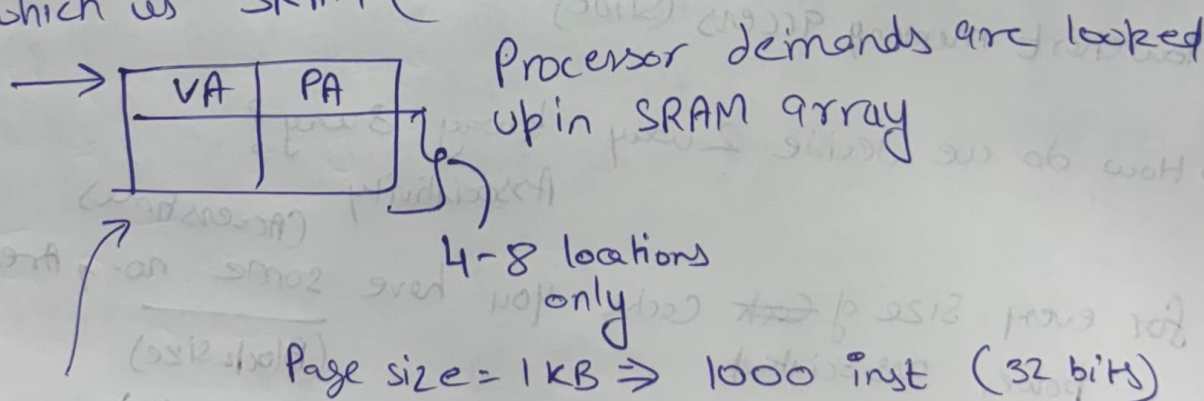
So, it is something non-cacheable, it must always be in memory



for Accessing any element, it will need two main memory access

- 1 - Page table
- 2 - page/instruction access

So we want to build fast memory using this as Ref. which is SRAM (for faster memory)



This structure is what we call TLB

It is in Critical path of processor hence generate faster translation for supporting memory
this we can extend Disk, memory, etc.
(idea)

Typically process assumes whole CPU is for them

Slides 3-4

Virtual memory, Address Translation

When we bring P_2 from P_1 , we need to invalidate the P.T

VA	PA	Valid	bit

if invalidated, then corresponding page table must be loaded

Basic idea (slide)

Overlap problem (slide)

Why you might need 4 KB page size? why not 1 KB?
32 bit inst we need 1000 inst

Size problem: swap space (slide)

Page table & TLB: (slide)

Flowchart memory access (slide)

→ How do we decide 2 way 4 way 8 way? Associativity

for every size of cache you have some no. of Assoc, power associated

(Access time)

(Block size)

(Cache capacity)

require a tool that estimates

Cactie — tool

(Best way) Model by using an RC circuit

Every wire, transistor, Capacitor, etc has RC Model
 everything modeled as RC circuit

Applying KVL on every single node and that's how you
 model cache which you then plug in to your Gem5

[Read about it from Smruti's book]

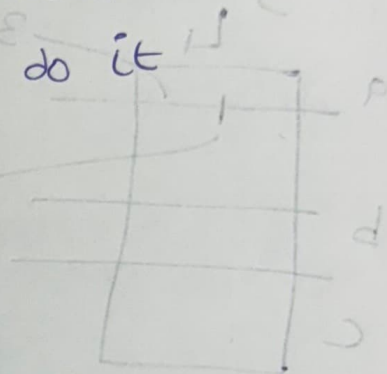
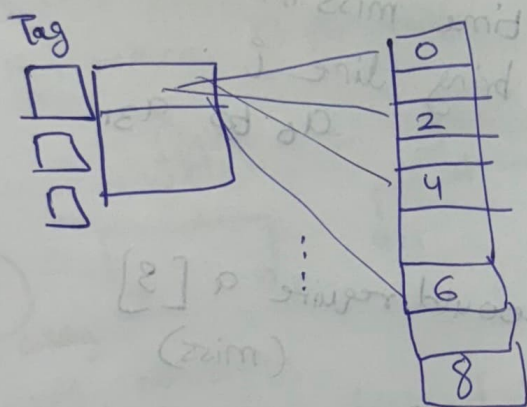
Pipelined cache

What do you pipeline?

is there something you can do in parallel

from architecture perspective can you do it

Cache indexes into array



So where does delay come?

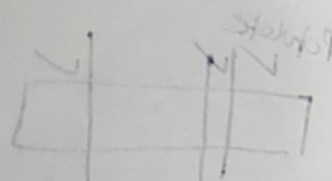
Key idea since there is capacitor out there, I need to

charge capacitor before Read/Write

so there is charging business and Tag Comparison

Step 1 Step 2 Step 3
 Block tag Data
 match Selection

→
 pipelined
 manner



• more on pipeline (slide)

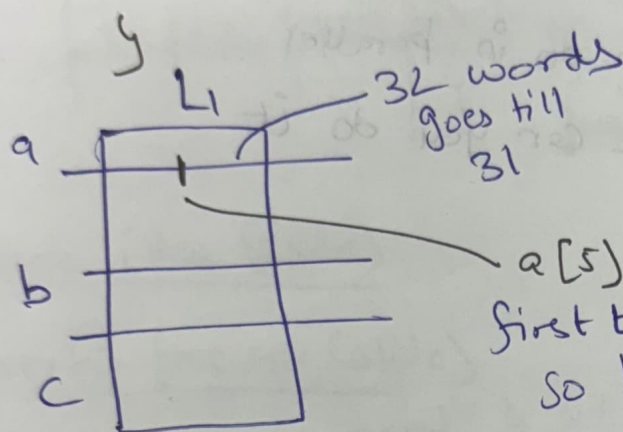
Other thing we like to take advantage of is that:-

for $(i = 4 \text{ --- } N)$

$$a(i+1) = b(i) + c(i)$$

$$b(i+1) = c(i+1) * 2$$

$$c(i+1) = a(i-2) + b(i-1) + c(i)$$



First time miss in cache
So bring line i
a0 to a31

So to compute $c[5]$ would require $a[8]$
(miss)

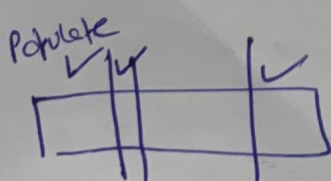
So it will again generate a miss

So you keep on generating these misses, while these

Another problem,

it can not even get bus, 1st miss is being generated
2nd is queued.

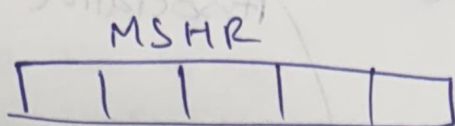
So in such cases you decouple the transition



(make non blocking
cache)

Anytime same address generating miss
You go and check if that miss is
Created earlier, if its created you just
wait

So we use Miss Status holding register (MSHR) values



(slide)

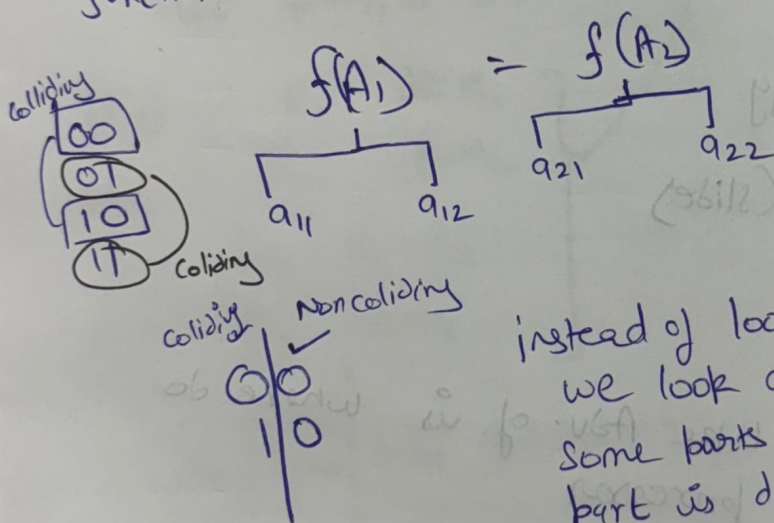
operation of an MSHR (slide)

Other problem we get is problem of collision

$$f(A_1) = f(A_2)$$

$$0 \begin{array}{|c|} \hline 1 \\ \hline \end{array} \% 2$$

function return 0 for every even Address



instead of looking whole address
we look at parts,
Some parts remain same other
part is different

$$f'(a_{11}) = f'(a_{21})$$

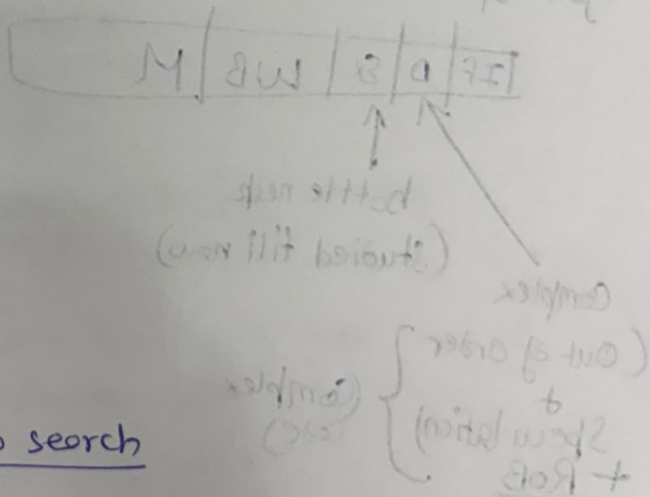
$$f'(a_{12}) = f'(a_{22})$$

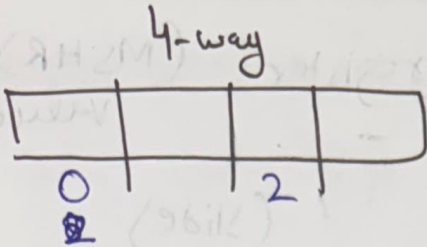
$$f'(a_{11}) \neq f'(a_{22})$$

(Basic insight slide)

Replacement - slide

Cuckoo search





Max we use 8 way
Associative Why?

Overhead req. here $\Rightarrow$

4 parallel comparisons
of tag

(Expensive)
(Takes Area)
(Latency, cycle time increases)

TLB on other hand is fully associative

So to improve we do prediction

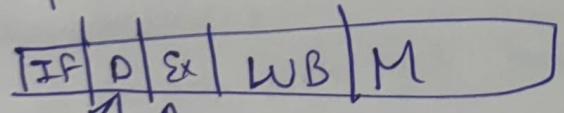
Way prediction (slide)

(Circuit must be in place)
 $\rightarrow$ saving energy

Way prediction Techniques (slide)

Loop tiling (slide)

Other thing we want to take Adv. of is where do
you spend most time in processor



(RISC)

bottle neck
(Studied till now)

Complex
(Out of order
+ Speculation)
+ ROB

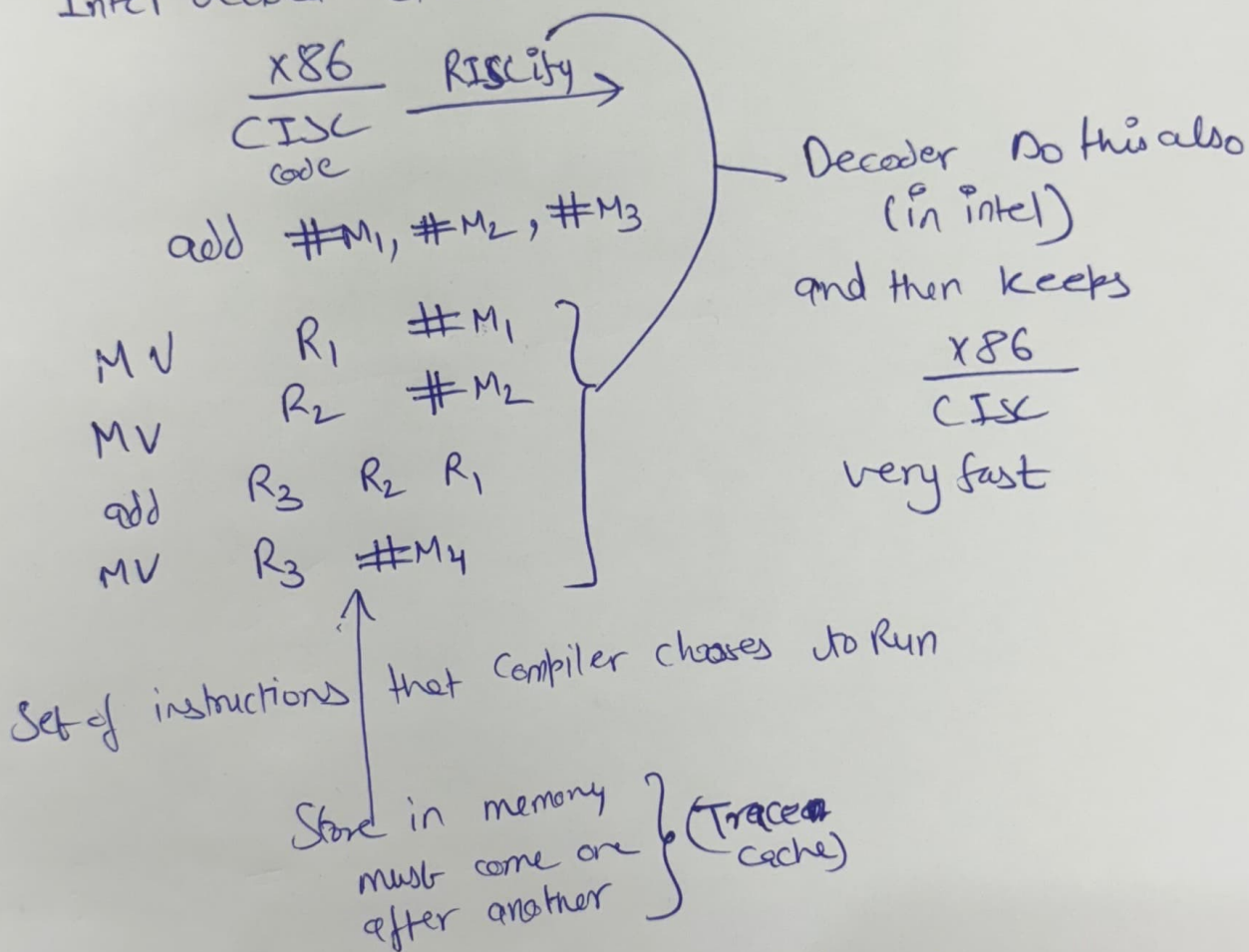
(Complex
Code)

RISC was invented because most of our prog required
Small set of inst to execution

- choose small No. of inst
- chose Load/store mechanism
- Execution via Reg.
- Instruction width - fixed
- Decoder is very small.

1-8 bytes instructions
(therefore decoder is complex)

Intel decided Execution happen in RISC manner



input to this come from Trace Cache bypassing memory

Can be done for seq. of inst
You execute often
(No locality of ref.)

Just execute & store lines you run
oftenly

(Stores after it on trace
cache)
on words

Preetch slides

Problem is with Data prefetch

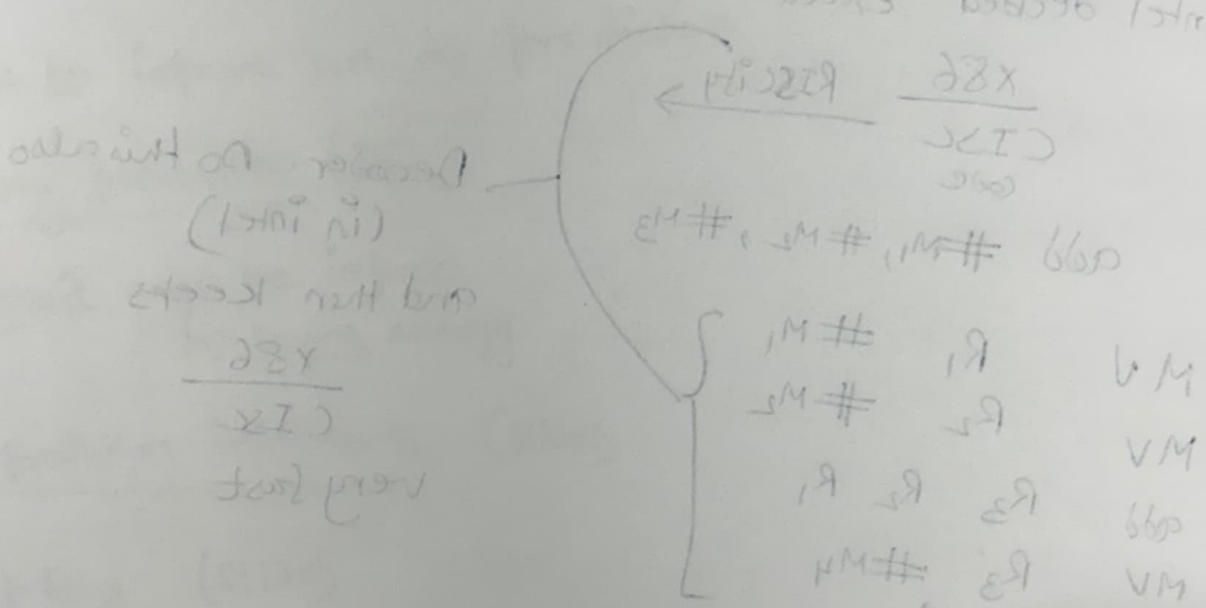
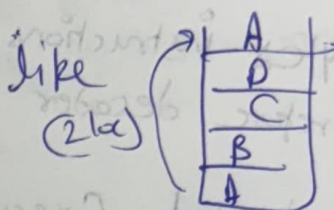
$X[4[i]]$

→ Allow stripmining, etc

(Slides)

Painter chasing (Also in assignment)

Allocation for next node might happen



Set of instructions that compiler chooses to run

Space in memory where code or instructions are stored

Input to the code from the previous instruction

Can be done for set of instructions

(locality of ref)

Just a cache of instructions